



TED UNIVERSITY

Department of Computer Engineering

MoonAI

Predator-Prey Neuroevolution Simulation Platform

Final Report

Team:

Caner Aras Emir İrkılata Oğuzhan Özkaya

Supervisor:

Dr. Ayşenur Birtürk

Jury Members:

Asst. Prof. Deniz Cantürk Asst. Prof. Mehmet Evren Coşkun

CMPE 492 — Senior Project II

Spring 2026

Table of Contents

1	Introduction	3
1.1	Project Motivation	3
1.2	Project Objectives	3
1.3	Project Context	4
1.4	Scope of This Report	4
2	Background, Related Work, and Research Inputs	4
2.1	Neuroevolution and Predator-Prey Simulation	4
2.2	Why a Synthetic Environment Matters	5
2.3	Related Engineering Inputs	5
2.4	Use of Library and Internet Resources	5
3	Requirements, Constraints, and Final Scope	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	6
3.3	Project Constraints	7
3.4	Final Scope Boundaries	7
4	Final System Architecture and Design	7
4.1	Architecture Overview	7
4.2	Module Decomposition	8
4.3	GPU-First Tick Flow	9
4.4	Design Rationale	10
5	Simulation Environment, Experiment Workflow, and Outputs	10
5.1	Experiment Definition and Parameter Space	10
5.2	Interactive Runtime Workflow	11
5.3	Selected-Agent Inspection and Observability	12
5.4	Output Artifacts and Analysis Pipeline	12
6	Implementation, Tools, and Technologies	13
6.1	Core Technology Stack	14
6.2	Why Rust and CUDA Were Used	14
6.3	Why the UI Stack Matters	14
6.4	Why the Analysis Stack Matters	15
6.5	Supporting Engineering Tools Used During the Course	15
7	Test Results	15
7.1	Executed Test Cases	16
7.2	Assessment of the Results	16
7.3	Bugs, Risks, and Potential Enhancements	17
8	Engineering Impact, Ethics, and Contemporary Issues	17
8.1	Global Impact	17
8.2	Economic Impact	17
8.3	Environmental Impact	18
8.4	Societal and Ethical Impact	18
8.5	Contemporary Issues Related to the Project Topic	18
8.6	Professional Responsibility	19

9	Final Project Status, Availability, and Future Work	19
9.1	Final Status Overview	19
9.2	Project Availability	19
9.3	Current Limitations	20
9.4	Future Work	20
9.5	Overall Final Assessment	20
10	Glossary	21
	References	22

1. Introduction

MoonAI is a simulation platform for studying evolutionary algorithms and neural network evolution through predator-prey dynamics. The project uses a synthetic ecosystem instead of a task-specific supervised dataset: predators, prey, and food interact in a configurable world; agent behavior is controlled by evolving neural networks; and the resulting runs produce both visual and structured data for later analysis.

The final deliverable is therefore not a single machine learning model with a fixed accuracy number. The main contribution is the simulation environment itself: a reproducible, interactive, GPU-accelerated benchmark for observing how neuroevolution behaves under changing ecological pressure. This emphasis is important because the project goal is to help study machine learning methods, not to claim that one evolved policy is the final answer to a real-world task.

1.1. Project Motivation

Modern machine learning experiments often depend on large labeled datasets or narrowly defined benchmarks. Those inputs are useful, but they also constrain what kinds of adaptation can be observed. MoonAI was motivated by a different question: can a self-generating environment be used to study how learning systems adapt, compete, diversify, and change over time?

The predator-prey setting is suitable for that purpose because it creates continuous selective pressure. Predators and prey are both required to survive, move, consume resources, and reproduce. Once those rules are combined with evolving neural networks and speciation, emergent behavior becomes an observable engineering artifact rather than an abstract concept.

1.2. Project Objectives

The project objectives were:

- implement a configurable simulation environment for predator-prey coevolution,
- integrate NEAT-style neuroevolution so both topology and weights can change over time [1],
- execute the hot simulation path on the GPU to keep large populations practical,
- provide an interactive desktop interface for experiment selection, live inspection, and settings control,
- generate reproducible output artifacts such as configuration snapshots, population metrics, species summaries, and representative genome exports,
- provide an analysis pipeline that turns run artifacts into a self-contained report.

1.3. Project Context

MoonAI was developed as a CMPE 492 senior design project. The work combines systems programming, GPU computing, simulation design, neuroevolution, UI engineering, data export, and technical documentation into a single project. The final codebase is a Rust and CUDA rewrite centered on a GPU-first runtime, with Python-based post-run analysis and a GitHub Pages documentation site.

The resulting system is useful in two ways. First, it is a research and teaching tool for studying evolutionary computation. Second, it is a software engineering case study in how to build a non-trivial simulation product around reproducibility, observability, and strong workflow discipline.

1.4. Scope of This Report

This final report covers:

- the background and design inputs that shaped the project,
- the final requirements, constraints, and scope boundaries,
- the final system architecture and simulation workflow,
- the tools and technologies used during the course,
- the executed test cases and their results,
- the engineering impact of the project in global, economic, environmental, and societal context,
- the final project status, availability links, limitations, and future work.

2. Background, Related Work, and Research Inputs

MoonAI sits at the intersection of neuroevolution, artificial life simulation, GPU computing, and interactive experiment tooling. The final design was shaped by both academic background and practical engineering references.

2.1. Neuroevolution and Predator-Prey Simulation

The underlying learning approach is NeuroEvolution of Augmenting Topologies (NEAT), which evolves both neural network weights and network structure over time [1]. NEAT is relevant to MoonAI because the project is concerned with adaptation under pressure, not only with parameter fitting inside a fixed model architecture.

Predator-prey worlds are a natural fit for this kind of study. They create coevolutionary pressure, local competition, and non-stationary behavior: the quality of a strategy depends on what other agents are doing at the same time. This makes the environment useful for observing emergent behavior, species diversity, and topology growth under changing conditions.

2.2. Why a Synthetic Environment Matters

The project deliberately prioritizes a synthetic environment over a pre-collected dataset. That decision has several research advantages:

- the environment generates its own behavior traces and metrics,
- experiments can be rerun with fixed seeds and controlled parameter sweeps,
- selective pressure is explicit and configurable,
- the benchmark can be extended without collecting external labeled data.

This does not make synthetic environments universally superior to real data. Instead, it makes them appropriate for a different class of questions: questions about adaptation, dynamics, policy diversity, and algorithmic behavior under repeated interaction.

2.3. Related Engineering Inputs

The final implementation was also shaped by practical design inputs rather than by a single prior system. The team relied on official documentation and mature open-source ecosystems for:

- Rust systems programming and package management [2],
- CUDA kernel programming and host-device orchestration [3],
- immediate-mode desktop UI and GPU rendering through `egui`, `eframe`, and `wgpu` [4, 5],
- Lua-based runtime configuration through `mlua` [6],
- JSON serialization and persistence through `serde` [7],
- post-run analysis and report generation through Pandas, Matplotlib, NetworkX, and Jinja2 [8, 9, 10, 11].

These sources were used to understand component behavior, data handling, rendering patterns, and reporting workflows. They informed the project architecture, but the final integration and runtime behavior are specific to MoonAI.

2.4. Use of Library and Internet Resources

Library and Internet resources were used in four main ways during the course:

1. to study background concepts such as NEAT and evolutionary computation,
2. to evaluate technology choices for GPU execution, UI rendering, serialization, and report generation,
3. to check component APIs, version compatibility, and workflow constraints,
4. to document and visualize the final system using GitHub Pages, LaTeX, and Mermaid diagrams [12].

The project therefore depends on both academic references and engineering documentation. The academic literature explains why the problem is interesting; the technical references

explain how the system can be implemented reliably.

3. Requirements, Constraints, and Final Scope

The final scope of MoonAI is defined by the requirements of a simulation platform, not by the requirements of a standalone production machine learning service. The central product is an environment for running, observing, and analyzing neuroevolution experiments.

3.1. Functional Requirements

The implemented system was designed to satisfy the following functional requirements.

- **FR-1:** The runtime shall load experiment definitions from `experiments.lua` at startup.
- **FR-2:** The runtime shall persist UI-related settings through `settings.json`.
- **FR-3:** The application shall allow users to select an experiment, edit a draft configuration, queue runs, and replace the active run from the UI.
- **FR-4:** The simulation shall evolve predator and prey agents whose controllers are neural networks with mutable topology and weights.
- **FR-5:** The runtime shall render the active population and provide selected-agent inspection data.
- **FR-6:** The runtime shall export structured artifacts including `config.json`, `stats.csv`, `species.csv`, and `genomes.json`.
- **FR-7:** The project shall include an analysis pipeline that reads run artifacts and generates a self-contained HTML report.
- **FR-8:** The runtime shall support fixed seeds so experiment results can be compared under controlled conditions.

3.2. Non-Functional Requirements

The most important non-functional requirements were:

- **Performance:** the hot simulation path should stay on the GPU so large populations remain practical.
- **Reproducibility:** experiment definitions, seeds, and output artifacts should make runs traceable and comparable.
- **Usability:** the project should provide a usable interface for selecting experiments, observing runs, and changing settings without editing source files.
- **Observability:** the runtime should expose enough metrics, snapshots, and profiler output to support debugging and analysis.
- **Maintainability:** configuration, UI, simulation, metrics, and analysis should be separated into clear modules.

3.3. Project Constraints

Several constraints strongly influenced the final design.

- **CUDA dependency:** the current runtime is centered on NVIDIA GPU execution.
- **Single-machine execution:** runs are executed locally, not through a distributed cluster scheduler.
- **Academic time budget:** the project had to prioritize a robust core runtime over optional platform features.
- **Verification cost:** GPU-first design improves runtime performance but makes step-by-step validation more demanding than a pure CPU reference implementation.
- **Documentation burden:** the project includes code, analysis tooling, documentation, and reports, so consistency across artifacts had to be managed carefully.

3.4. Final Scope Boundaries

The final scope boundaries are as important as the implemented features.

- MoonAI is **not** primarily a project about maximizing one machine learning output metric. Its main focus is the environment that supports study of evolutionary learning.
- MoonAI is **not** a generic reinforcement learning framework, cloud platform, or distributed experiment manager.
- MoonAI is **not** a production scientific cluster tool; it is a research-oriented, interactive simulation product with supporting analysis utilities.

These boundaries kept the implementation focused on one coherent goal: a high-performance simulation environment for studying neuroevolution through predator-prey dynamics.

4. Final System Architecture and Design

The final MoonAI architecture follows a GPU-first design. The CPU loads experiments and settings, starts the UI, orchestrates kernel launches, and writes compact exports. The GPU owns the simulation state and performs the dominant computation for sensing, inference, movement, combat, reproduction, and metrics reduction.

4.1. Architecture Overview



Figure 1: High-level architecture of the final MoonAI runtime and analysis workflow.

As shown in Figure 1, the runtime begins with two external inputs: `experiments.lua` and `settings.json`. The executable loads those files, initializes the desktop application, creates or replaces run sessions, and coordinates GPU simulation plus output logging. Completed run artifacts are then consumed by the Python analysis pipeline to produce a self-contained HTML report.

4.2. Module Decomposition

The final repository is organized around clear module responsibilities.

Table 1: Main implementation modules in the final codebase

Module	Responsibility
<code>main.rs</code>	Bootstraps the executable, resolves the runtime directory, loads settings and experiments, and starts the UI application.
<code>experiment.rs</code>	Defines <code>SimulationConfig</code> , loads <code>experiments.lua</code> , injects defaults into Lua, and validates experiment values.
<code>settings.rs</code>	Loads and saves <code>settings.json</code> and defines the persistent UI configuration surface.
<code>src/ui/</code>	Implements the desktop application shell, run queue, active run session, render helpers, UI state, and custom world renderer.
<code>src/sim/</code>	Defines the host-side simulation API, GPU readback structures, and CUDA bindings for the simulation and evolution kernels.
<code>metrics.rs</code>	Writes structured runtime artifacts to the experiment output directory.
<code>analysis/</code>	Discovers completed runs, groups conditions, generates plots, and renders a self-contained HTML analysis report.

This decomposition keeps configuration, runtime execution, rendering, metrics export, and analysis separate enough to remain understandable while still functioning as one coherent system.

4.3. GPU-First Tick Flow

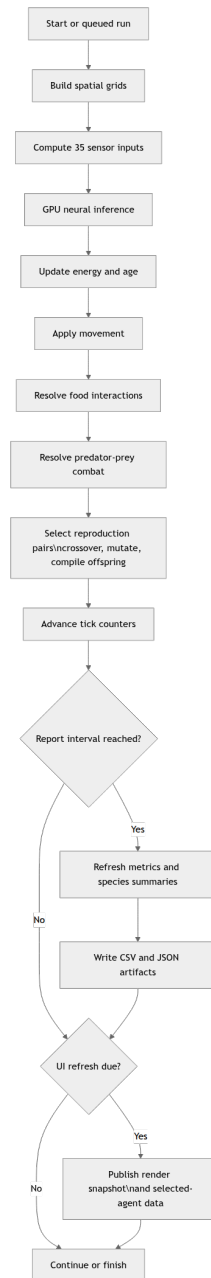


Figure 2: Simplified tick flow for the final GPU-first simulation runtime.

The internal tick execution path is summarized in Figure 2. At each step, the runtime builds spatial grids, computes sensor inputs, executes neural inference, updates agent state, resolves interactions, performs reproduction, advances counters, and optionally refreshes metrics and render snapshots.

Two cadences are intentionally separated in the design:

- **report_interval_ticks** controls when export artifacts such as `stats.csv`, `species.csv`, and `genomes.json` are refreshed.

- **UI speed multiplier** controls how often the interactive interface refreshes the visible world state.

This separation matters because reporting and rendering serve different purposes. Export should remain consistent and comparable across runs, while UI refresh rate should remain adjustable for live observation.

4.4. Design Rationale

The main architectural decisions were driven by three principles:

1. **Keep the hot path on the GPU.** Large populations make CPU-side simulation unattractive. GPU ownership of simulation state reduces host-side duplication and keeps the main path compact [3].
2. **Expose the system through a usable interface.** The project is not only a kernel benchmark. It is also a tool for selecting experiments, watching live behavior, inspecting agents, and exporting results through a desktop application built with `egui` and `wgpu` [4, 5].
3. **Make every run analyzable after execution.** Structured metrics, species summaries, and representative genomes are part of the product, not optional extras.

This combination of GPU execution, interactive runtime control, and post-run analysis is the core of the final MoonAI design.

5. Simulation Environment, Experiment Workflow, and Outputs

MoonAI is centered on the simulation environment itself. The final runtime defines how experiments are described, how they are executed, what data is produced, and how that data is reviewed after the run.

5.1. Experiment Definition and Parameter Space

The base experiment structure is defined by `SimulationConfig`. The default configuration includes a square world, separate predator and prey populations, food entities, energy rules, mutation rates, compatibility parameters, reporting cadence, and seed control. Current defaults include a world size of 3000, 12000 predators, 48000 prey, 60000 food items, and a report interval of 1000 ticks.

The shipped `experiments.lua` file expands that base into a broad experiment matrix. It defines 55 named conditions and applies 5 fixed seeds to each, producing 275 seeded runs, plus an additional `default` entry for ad hoc use. The experiment groups cover multiple kinds of study rather than only one benchmark dimension.

Table 2: Main experiment groups defined in `experiments.lua`

Group	Main variable	Purpose
Baseline sweeps	Mutation, speciation, population size	Compare broad evolutionary behavior under different default-scale pressures.
Scale experiments	Population and world size	Observe how the runtime and population dynamics behave as density and total counts change.
Density experiments	World size at fixed population	Study sparse and dense ecological pressure at the same agent count.
Run-length experiments	<code>max_ticks</code>	Compare short and long runs under otherwise similar conditions.
Energy/resource experiments	Food respawn and energy budget	Stress-test survival and reproduction under scarce or abundant resources.
Perception and mobility	Vision, speed, interaction range	Observe how sensing and movement shape survival strategies.
Complexity experiments	Add-node, add-connection, hidden-node limits	Study how network growth constraints affect behavior and topology.

5.2. Interactive Runtime Workflow

The application always launches the UI. From the interface, a user can select a preset, edit a draft configuration, queue runs, replace the current run, inspect the world, and adjust visual settings without modifying source files.

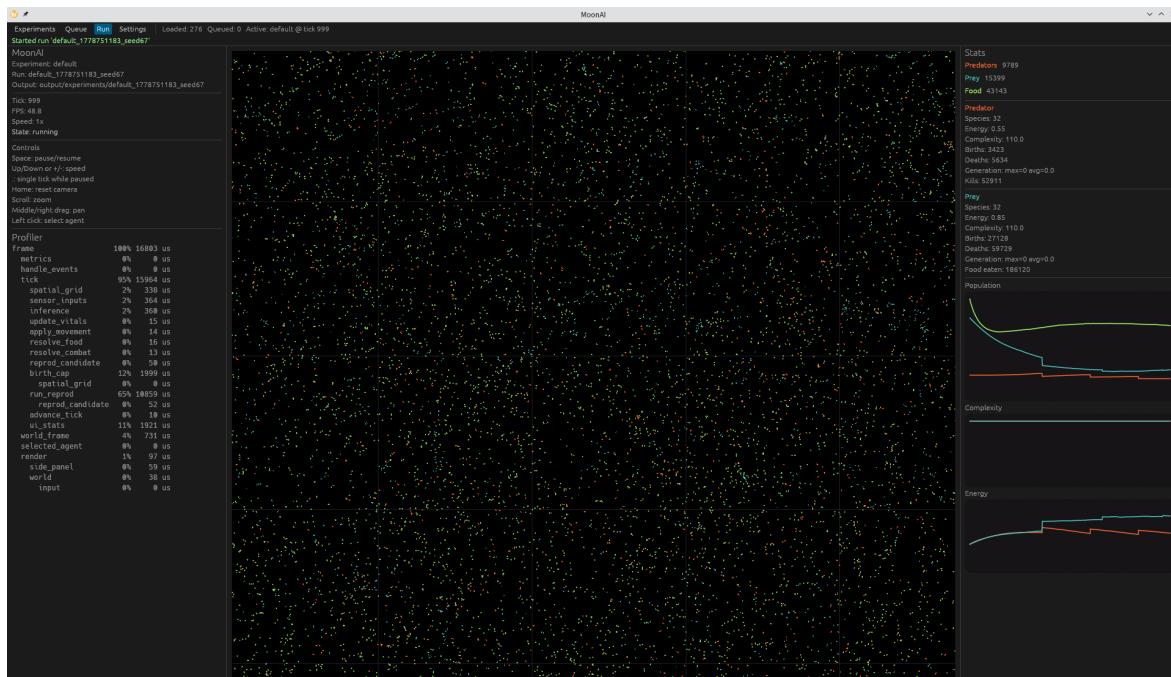


Figure 3: MoonAI desktop application showing the live simulation interface and experiment inspection workflow.

The runtime view shown in Figure 3 reflects several design decisions:

- the full active population is rendered instead of a sampled subset,
- a selected agent can expose additional information such as sensor lines and neural topology,
- queue management is built into the application shell,
- settings are persisted separately from experiment definitions.

This separation between experiment definitions and UI state is important for repeatability. Simulation parameters live in `experiments.lua`, while interface settings live in `settings.json`.

5.3. Selected-Agent Inspection and Observability

MoonAI does more than display moving points. When an agent is selected, the UI can show a vision circle, sensor lines, a stats panel, and a network panel. The controller receives 35 inputs consisting of nearest predators, nearest prey, nearest food items, self state, and wall proximity. This makes the runtime useful not only for entertainment or visualization, but for actual inspection of how evolved behavior is being produced.

The runtime also includes a scoped profiler tree. This profiler is useful when evaluating where time is spent across spatial grid construction, movement, reproduction, and other host-side control scopes.

5.4. Output Artifacts and Analysis Pipeline

Every run writes a structured output bundle under `output/experiments/`. The most important files are listed below.

Table 3: Primary experiment output artifacts

File	Purpose
<code>config.json</code>	Full configuration snapshot for the executed run.
<code>stats.csv</code>	Time-series population and complexity metrics at each report interval.
<code>species.csv</code>	Species-level population summaries over time.
<code>genomes.json</code>	Representative genome snapshots including nodes and connections.

Those artifacts feed the Python analysis package. The analysis pipeline discovers valid runs, groups them by condition, builds comparison plots, renders representative genomes, and writes a self-contained HTML report.

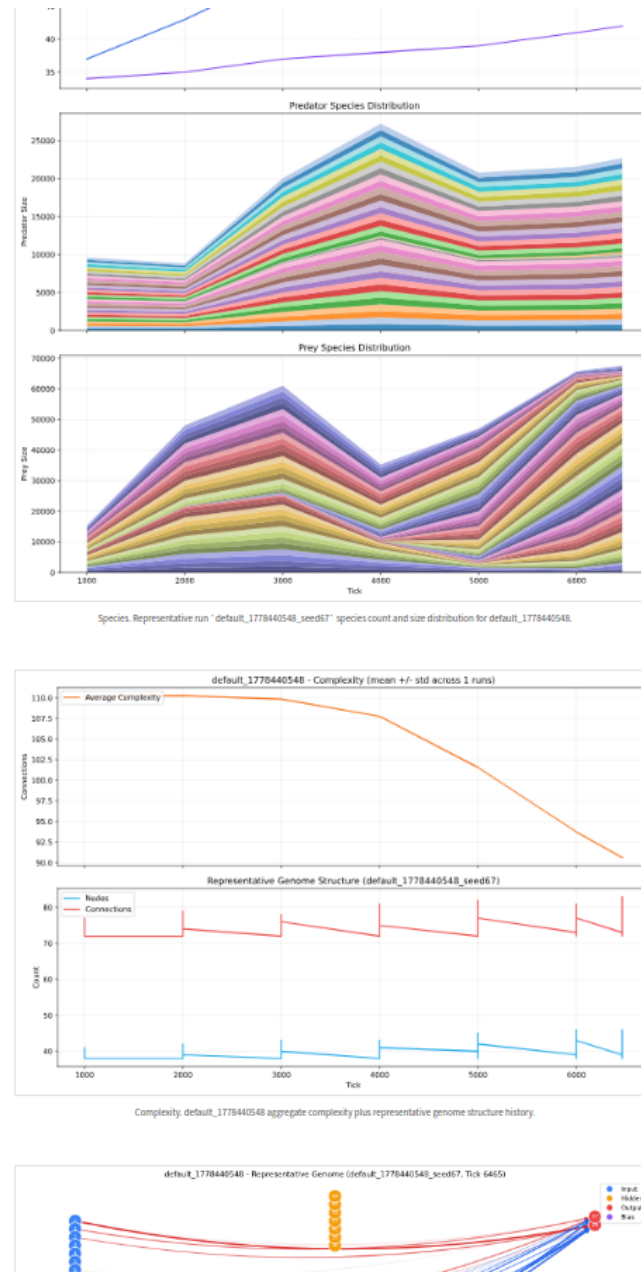


Figure 4: Generated HTML analysis report used to compare experiment conditions and summarize completed runs.

The report shown in Figure 4 is an important part of the final system. It turns raw exports into a usable comparison surface for researchers, which reinforces the project's main goal: providing an environment for studying evolutionary machine learning, not merely running one simulation loop.

6. Implementation, Tools, and Technologies

The final MoonAI codebase combines systems programming, GPU execution, UI rendering, serialization, experiment analysis, and documentation tooling. The value of this section is not to list technologies mechanically, but to explain why they were used.

6.1. Core Technology Stack

Table 4: Main tools and technologies used in the final implementation

Layer	Technology	Role in MoonAI
Systems language	lan- Rust 2024	Primary implementation language for the runtime, UI, configuration, metrics, and analysis integration points [2].
GPU execution	CUDA via build.rs and cc	Executes the hot simulation, evolution, and readback path on NVIDIA GPUs [3].
Desktop UI	eframe/egui	Provides the application shell, controls, panels, and interactive runtime workflow [4].
GPU rendering	wgpu	Renders the world view and supports custom population drawing in the UI [5].
Runtime configuration	mlua	Loads <code>experiments.lua</code> so simulation presets remain editable without recompilation [6].
Persistence	serde / serde_json	Stores settings and exports structured artifacts in JSON form [7].
Analysis	Pandas, Matplotlib, NetworkX, Jinja2	Loads run artifacts, computes summaries, renders plots and genome diagrams, and builds self-contained HTML reports [8, 9, 10, 11].
Workflow	just, uv	Standardizes build, test, and analysis commands for the project workflow.
Documentation	Zensical, GitHub Pages, Mermaid, LaTeX	Publishes docs, renders diagrams, and produces course reports [12].

6.2. Why Rust and CUDA Were Used

Rust was selected because the final system needed strong module organization, explicit data types, good tooling, and direct interoperability with low-level code. The CUDA side was needed because the project is fundamentally population-heavy: sensor computation, inference, movement, reproduction, and compact readback become expensive when tens of thousands of agents are simulated simultaneously.

The combination allows MoonAI to keep high-level application flow readable while still moving the computationally dominant path to the GPU.

6.3. Why the UI Stack Matters

The UI stack is not cosmetic. It is part of the engineering solution. `egui` and `wgpu` were chosen because the project required:

- a desktop application with live controls,
- custom rendering for a dense simulated world,
- selection and inspection of individual agents,

- in-application queue management and settings editing.

This turns MoonAI from a raw simulation executable into an actual experimental workbench.

6.4. Why the Analysis Stack Matters

The Python analysis stack was selected to support post-run comparison rather than real-time simulation. Pandas is used for structured metric handling, Matplotlib for plots, NetworkX for topology visualization, and Jinja2 for self-contained HTML report rendering. This separation of concerns is pragmatic: Rust and CUDA handle the runtime, while Python handles data analysis and presentation efficiently.

6.5. Supporting Engineering Tools Used During the Course

Several supporting tools were also important during the course:

- **just** standardized recurring commands such as build, test, and analysis.
- **uv** simplified Python environment and dependency management for the analysis tooling.
- **Mermaid CLI** was used to generate architecture diagrams for the final report and documentation [12].
- **GitHub Pages** and Zensical were used to publish project documentation and report links.
- **LaTeX** was used to prepare the formal design and final course reports.

Together, these tools made the project easier to build, reason about, validate, and present.

7. Test Results

The final test results were recorded against the current Rust and CUDA implementation. During the report preparation process, the automated regression command `just test` was re-run on the repository snapshot and completed successfully with 7 passing automated tests and 0 failures.

Because the implementation changed substantially during the course, the final reported cases below were traced to the current codebase and final project artifacts rather than copied from an older implementation plan.

7.1. Executed Test Cases

Table 5: Final executed test cases and outcomes

ID	Test description	Result	Evidence and notes
TR-01	Metrics logger writes expected experiment artifacts.	Pass	Automated unit test in <code>src/metrics.rs</code> ; confirms <code>stats.csv</code> , <code>species.csv</code> , <code>config.json</code> , and <code>genomes.json</code> .
TR-02	Profiler records nested scope trees correctly.	Pass	Automated unit test in <code>src/profiler.rs</code> ; confirms hierarchical scope accounting.
TR-03	Profiler output rows preserve indentation and formatting rules.	Pass	Automated unit test in <code>src/profiler.rs</code> ; confirms readable profiler formatting.
TR-04	Vision radius matches world-to-screen projection.	Pass	Automated unit test in <code>src/ui/render.rs</code> ; checks selected-agent overlay math.
TR-05	Predator and prey triangle geometry remains normalized.	Pass	Automated unit test in <code>src/ui/world.rs</code> ; checks renderer geometry scaling.
TR-06	Agent selection radius stays within configured bounds.	Pass	Automated unit test in <code>src/ui/world.rs</code> ; checks selection interaction limits.
TR-07	Food size setting maps correctly into world-space rendering.	Pass	Automated unit test in <code>src/ui/world.rs</code> ; confirms UI setting propagation.
TR-08	GUI runtime launches and renders a live experiment view.	Pass	Final screenshot evidence in Figure 3.
TR-09	Analysis report generation produces a usable HTML summary.	Pass	Final screenshot evidence in Figure 4.

7.2. Assessment of the Results

The executed final test set contains 9 recorded cases:

- 7 automated unit tests,
- 2 manual smoke checks backed by final project artifacts,
- 9 passing cases,
- 0 failing cases.

This result is strong enough to support the final submission, but it also reveals where future validation work should improve. The current automated coverage is best around metrics export, profiler formatting, and render math. Coverage is still lighter for end-to-end GPU simulation behavior, long-duration stability, and broader system smoke automation.

7.3. Bugs, Risks, and Potential Enhancements

No failing test case remained in the rerun final snapshot. However, several verification risks remain relevant:

- end-to-end kernel invariants are harder to test than pure host-side logic,
- the UI workflow still relies partly on manual validation,
- the current repository does not yet include a broad automated matrix for complete build, run, and artifact checks across multiple platforms.

The most useful testing enhancements for future work are:

1. add deterministic seed-comparison tests for compact GPU readbacks,
2. add artifact-schema regression tests for completed runs,
3. add automated smoke tests for representative experiment conditions,
4. extend manual test evidence into repeatable scripted system checks.

8. Engineering Impact, Ethics, and Contemporary Issues

MoonAI is not only a technical artifact. It also has broader engineering implications because it changes how machine learning behavior can be studied, compared, and reproduced.

8.1. Global Impact

At a global level, MoonAI contributes to a broader movement toward synthetic environments and reproducible AI experimentation. A system that can generate its own evolving behavioral data is valuable for research groups that do not have access to large real-world datasets or expensive data collection pipelines.

The project also shows that meaningful machine learning research infrastructure does not always mean collecting more external data. In some settings, building a better environment is the more important engineering contribution.

8.2. Economic Impact

The project was intentionally built on open-source tools and libraries. That has positive economic effects:

- lower software licensing cost,
- easier reuse by future students or research teams,
- lower onboarding cost because the stack is based on widely documented tools.

However, the project also makes a real economic tradeoff visible: high-performance simulation still depends on capable GPU hardware. Open-source software reduces licensing barriers, but it does not eliminate compute cost.

8.3. Environmental Impact

The environmental impact of MoonAI is mixed. On one hand, simulation can reduce the need for repeated physical field trials or repeated collection of external behavioral data. On the other hand, GPU-heavy workloads consume non-trivial energy.

This means performance engineering has environmental consequences as well. Efficient kernels, compact readback, and avoiding unnecessary duplicate computation are not only speed concerns; they are also responsible engineering choices.

8.4. Societal and Ethical Impact

MoonAI has a positive educational and research impact because it provides a controlled environment for studying learning behavior. It allows experimentation without collecting personal data by default, which is a meaningful advantage compared with many real-world AI datasets.

At the same time, synthetic environments create a different ethical risk: overclaiming what a simulation result means. A strategy that performs well inside one benchmark should not be misrepresented as proof of general intelligence or guaranteed real-world behavior.

This is consistent with professional engineering responsibilities emphasized by ACM and IEEE ethics guidance [13, 14]:

- describe capabilities accurately,
- disclose limitations clearly,
- avoid overstating conclusions from partial evidence,
- build tools that support understanding rather than confusion.

8.5. Contemporary Issues Related to the Project Topic

Several contemporary issues are directly relevant to MoonAI.

- **Reproducibility in AI research:** many results remain hard to repeat. MoonAI addresses this through explicit experiment definitions, seeds, and structured outputs.
- **Simulation-to-reality gap:** synthetic environments are powerful, but their conclusions must be interpreted carefully.
- **Compute concentration:** advanced experimentation often depends on hardware that is not equally available to all researchers.
- **Open-source governance:** research software depends on external open-source components, so versioning, documentation quality, and long-term maintenance matter.
- **Explainability of adaptive systems:** as evolved behavior becomes more complex,

observability tools such as profiler views, species summaries, and network inspection become more important.

8.6. Professional Responsibility

The final engineering lesson is straightforward: building the environment, documenting it, and reporting its limitations are part of the solution. For MoonAI, responsible engineering means providing a useful experimental platform while remaining honest about what the platform does and does not prove.

9. Final Project Status, Availability, and Future Work

The final MoonAI project reached a complete prototype stage. The main runtime, UI, experiment catalog, metrics exports, and analysis pipeline are all implemented in the repository. The project is therefore finished as a senior design deliverable, while still leaving room for future research and productization work.

9.1. Final Status Overview

Table 6: Final status by work package

Work package	Status	Notes
GPU-first simulation runtime	Complete	Core tick execution, readback, and evolution path are implemented.
Interactive desktop application	Complete	Experiment selection, queueing, run replacement, settings control, and live inspection are implemented.
Experiment configuration system	Complete	Runtime loading from <code>experiments.lua</code> and validation through <code>SimulationConfig</code> are implemented.
Output artifact generation	Complete	Configuration, metrics, species, and genome exports are implemented.
Analysis and reporting pipeline	Complete	HTML analysis generation from run artifacts is implemented.
Documentation website	Complete	GitHub Pages site and project documentation are available.
Broad end-to-end automation	Partial	Automated unit coverage exists, but wider system smoke automation can still be improved.

9.2. Project Availability

The project is publicly available through both its source repository and its documentation site.

Table 7: Project URLs required for the final report

Item	URL
GitHub repository	https://github.com/moon-aii/moonai
GitHub Pages documentation	https://moon-aii.github.io/moonai/

These links provide access to the source code, documentation, and report artifacts published by the team.

9.3. Current Limitations

The final project still has several clear limitations.

- The runtime currently depends on CUDA-capable NVIDIA hardware for its intended hot-path execution model.
- The automated test suite is useful, but still lighter on full end-to-end GPU validation than on host-side logic.
- The project is optimized for local execution and study, not for distributed experiment scheduling or multi-user orchestration.
- The project’s main focus is the simulation environment, so conclusions about the quality of individual evolved behaviors must remain conservative.

9.4. Future Work

The most valuable future extensions are:

1. add stronger end-to-end GPU invariant and determinism tests,
2. extend artifact comparison and regression tooling across experiment batches,
3. improve automation for long-running experiment suites,
4. expand interactive analysis features for selected agents and species trends,
5. investigate additional evolutionary strategies and environment variants on top of the current runtime.

9.5. Overall Final Assessment

The overall final assessment of MoonAI is positive. The project successfully delivered a high-performance simulation platform for studying neuroevolution in a predator-prey world, along with an interactive UI, structured artifacts, and analysis tooling. The final result is a useful experimental environment and a credible engineering capstone product.

10. Glossary

Table 8: Key terms used in the report

Term	Meaning
NEAT	NeuroEvolution of Augmenting Topologies, the evolutionary algorithm used to evolve both weights and neural network structure.
Genome	The encoded neural-network representation of an agent, including node and connection genes.
Species	A compatibility-based grouping of similar genomes used to preserve innovation during evolution.
Readback	Copying compact simulation state from GPU memory to host memory for UI or export.
Report interval	The tick cadence at which structured output artifacts are refreshed and written.
Speed multiplier	The UI-side visualization cadence that controls how many simulation ticks are advanced per visible frame.
Render snapshot	The compact host-visible population data used to draw the world in the UI.
Run queue	The UI-managed list of pending experiments that execute sequentially.
Representative genome	A selected genome snapshot exported to describe one species or agent at a reporting point.
GPU-first architecture	A design in which the dominant simulation state and hot execution path remain on the GPU, while the CPU mainly orchestrates and exports compact results.

References

- [1] K. O. Stanley and R. Miikkulainen, “Evolving Neural Networks through Augmenting Topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] The Rust Project Developers, *The Rust Programming Language*, 2026. [Online]. Available: <https://www.rust-lang.org/>
- [3] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [4] Egui Contributors, *egui / eframe*, 2026. [Online]. Available: <https://github.com/emilk/egui>
- [5] wgpu Contributors, *wgpu*, 2026. [Online]. Available: <https://wgpu.rs/>
- [6] mlua Contributors, *mlua*, 2026. [Online]. Available: <https://github.com/mlua-rs/mlua>
- [7] Serde Contributors, *Serde*, 2026. [Online]. Available: <https://serde.rs/>
- [8] Pandas Development Team, *pandas*, 2026. [Online]. Available: <https://pandas.pydata.org/>
- [9] Matplotlib Development Team, *Matplotlib*, 2026. [Online]. Available: <https://matplotlib.org/>
- [10] NetworkX Developers, *NetworkX*, 2026. [Online]. Available: <https://networkx.org/>
- [11] Pallets Projects, *Jinja*, 2026. [Online]. Available: <https://jinja.palletsprojects.com/>
- [12] Mermaid Contributors, *Mermaid*, 2026. [Online]. Available: <https://mermaid.js.org/>
- [13] Association for Computing Machinery, *ACM Code of Ethics and Professional Conduct*, 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [14] IEEE, *IEEE Code of Ethics*, 2020. [Online]. Available: <https://www.ieee.org/about/corporate/governance/p7-8.html>